

Text-Guided Image Retrieval System

Introduction :

Retrieving images based on text descriptions is a significant challenge in computer vision and language processing. As digital search engines and content libraries grow, the need for accurate text-to-image matching becomes even more important. Traditional methods relying on metadata or manual tagging often fall short when queries are vague or require deeper understanding. This project leverages the CLIP model to bridge the gap between vision and language, enabling the retrieval of more relevant images by representing both text and images in a shared space.

Project Overview :

This project focuses on the development of a Text-Guided Image Retrieval System utilizing the CLIP (Contrastive Language-Image Pretraining) model developed by OpenAI. The system aims to retrieve images that are semantically relevant to a given textual query. It achieves this by encoding both text and image data into multi-modal embeddings, which are then used to calculate similarity scores between the text input and image candidates. The performance of the system is evaluated using various metrics, including Precision, Recall, F1 Score, and Hit@1. The results demonstrate the effectiveness of the CLIP model in retrieving relevant images based on textual descriptions, highlighting its potential applications in search engines, content-based image retrieval, and multimedia systems.

Objectives:

The goals of this project are :

- To develop an efficient text-guided image retrieval system that can return relevant images based on textual queries.
- To evaluate the retrieval system using standard retrieval metrics such as Precision, Recall, F1, and Hit@1.
- To explore the capabilities of the CLIP model in handling cross-modal search tasks involving images and text.
- To assess the system's performance on both validation and test sets.
- Create a user-friendly interface for retrieving images based on text.

Used Platform:

- Google Colab and Kaggle Notebook for interactive coding and testing.

Methodology :

System Overview

The system operates by taking a text query as input, encoding it into a fixed-length vector (embedding) using the CLIP model. It then compares this text embedding with precomputed image embeddings using similarity measures such as cosine similarity. The top-K most similar images are retrieved based on these comparisons.

The flowchart below illustrates the key steps involved in the system's operation:

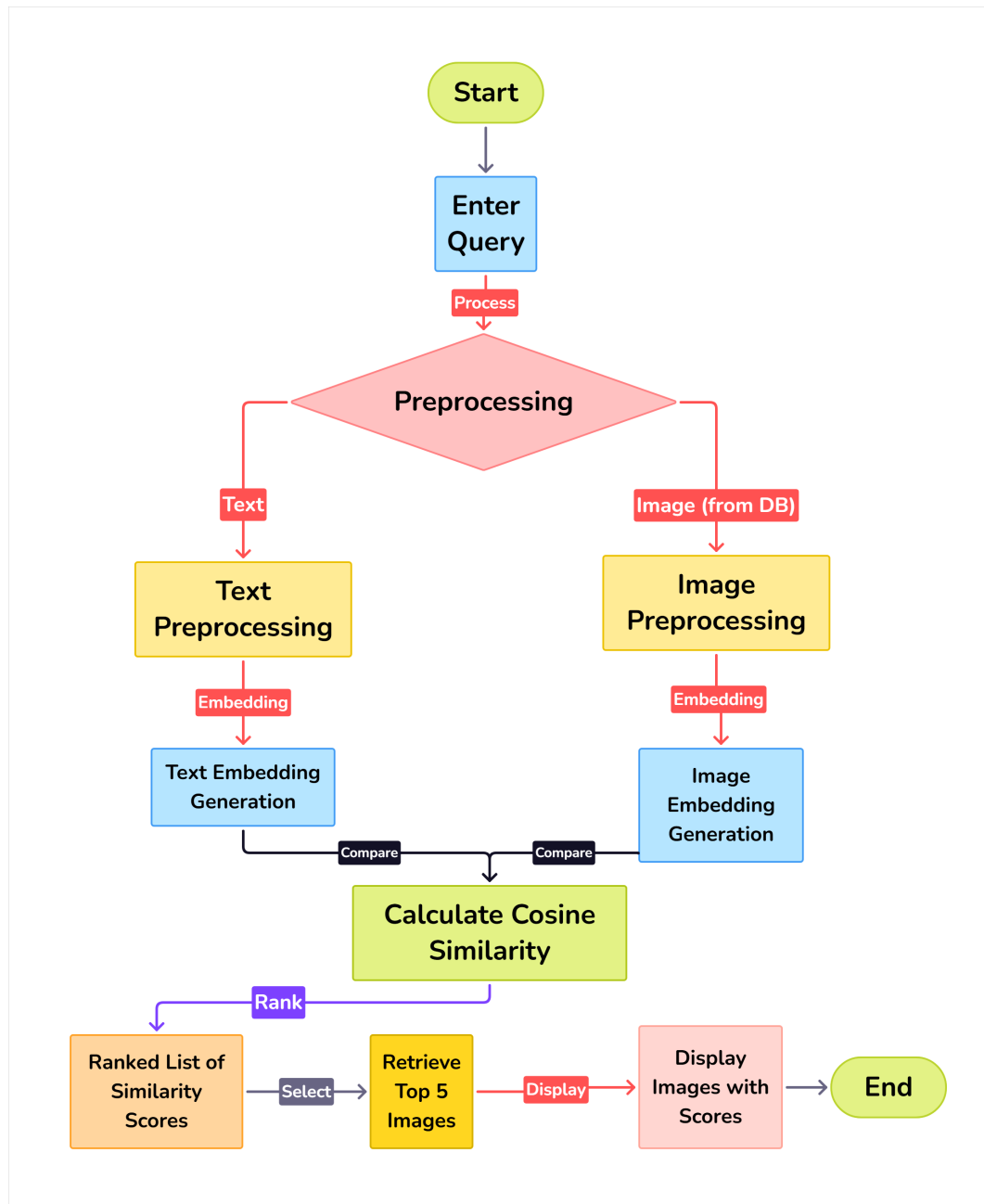


Figure 1: Flowchart of the Text-Guided Image Retrieval System

Step 1 - Data Preparation :

Import Libraries & Device Setup

Google Drive was mounted to allow access to project files. Then, the necessary libraries were imported to set up the environment for the project.

Libraries Used:

- **pandas**: Was used for data manipulation and analysis.
- **numpy**: Was used for array operations and numerical calculations.
- **torch**: Was used as a deep learning framework for tensor computations.
- **os**: Was used for interacting with the operating system (e.g., file paths).
- **torch.utils.data**: Was used to provide utilities for creating datasets and loading batches.
- **PIL (Python Imaging Library)**: Was used for opening, manipulating, and saving image files.
- **io, ast**: Were used for handling input/output operations and working with string representations of data.
- **CLIPProcessor & CLIPModel**: Were used from HuggingFace to process text and images for the CLIP model.
- **scikit-learn**: Was used for splitting datasets and calculating evaluation metrics.
- **tqdm**: Was used as a progress bar library for tracking loops.
- **scipy.spatial.distance**: Was used for calculating cosine similarity between vectors.
- **IPython.display**: Was used for displaying output in Jupyter/Colab notebooks.
- **matplotlib**: Was used for plotting visualizations and graphs.
- **torch.nn.functional**: Was used to provide functions for neural network operations (e.g., activation functions, loss functions).

Setting Device:

- The device was set to **GPU** if available, otherwise, it was set to **CPU**. This ensured the system ran on the most efficient hardware available for deep learning tasks, speeding up training and inference when using a GPU.

Load Dataset

The **Pokemon Dataset** was used from [Kaggle](#). The dataset was loaded from the working directory. If the dataset existed at the given location, it was read into a pandas DataFrame using `pd.read_csv()`. The first few rows of the dataset were displayed to verify that the data had been loaded successfully.

Convert Byte Strings to Images

- Byte strings representing images were converted into actual image objects. The byte string data was first loaded into memory, and then it was read and interpreted as an image file.
- The images were resized to 224x224 pixels to ensure consistency in input dimensions for the model.

Purpose: This step was crucial for converting the image data stored in byte format into a usable image format for the model.

Split Dataset

- The dataset was divided into three sets: training, validation, and testing. Initially, 30% of the data was separated for validation and testing. This portion was then equally split into two parts for validation and testing.
- The sizes of the resulting sets were printed to confirm that the data was split as intended.

Purpose: This step was important for ensuring the model was trained on one portion of the data and evaluated on separate, unseen data, which helps in reducing overfitting.

Custom PyTorch Dataset for CLIP (Image-Text Pairs)

- A custom dataset class was created to handle image-text pairs. The dataset was prepared by processing the images and text, which were then converted into the appropriate format for use with the CLIP model.
- The dataset was structured to return both the image data and its corresponding text description, ensuring that each pair could be processed and used by the model for training and evaluation.
- An indexing system was implemented to track the data, allowing efficient access and manipulation of the dataset during model training and testing.

Purpose: This step was essential for creating a dataset that could be used by the CLIP model, ensuring the system could efficiently process and utilize both text and image data for training.

DataLoader with Custom Collate Function

- A custom `collate function` was created to handle the batching of images and text data. This function stacked the pixel values, input IDs, attention masks, and indices for each item in the batch. The sequences were padded to a uniform length, ensuring consistency across batches.
- The custom collate function was then applied to the `DataLoader` to load the training, validation, and test datasets in batches of size 16.

Purpose: This step was crucial for preparing the data into batches that could be efficiently processed by the model, ensuring proper handling of sequences and padding during training and evaluation.

Step 2 - Model Setup :

Load CLIP Model and Processor

The `CLIPProcessor` and `CLIPModel` were loaded using the pre-trained "openai/clip-vit-base-patch32" model. The model was moved to the selected device (GPU or CPU) to ensure efficient execution.

Fine-Tuning CLIP with Contrastive Learning

- The model was fine-tuned using *contrastive learning*, where both image and text features were learned simultaneously. The `AdamW` optimizer was used with a learning rate of 10^{-5} . The training was performed over three epochs, and the loss was calculated as the sum of the cross-entropy loss for both image and text features, followed by normalization.
- For each batch, the image and text features were extracted using the CLIP model, normalized, and passed through the loss function. The optimizer was updated, and the running loss was printed to track the progress of the training.

```
Training Epoch 1: 100%|██████████| 37/37 [00:35<00:00, 1.04it/s]
Epoch 1 Loss: 2.4056
Training Epoch 2: 100%|██████████| 37/37 [00:32<00:00, 1.14it/s]
Epoch 2 Loss: 2.1357
Training Epoch 3: 100%|██████████| 37/37 [00:31<00:00, 1.16it/s]
Epoch 3 Loss: 2.0479
Fine-tuned CLIP model saved.
```

Figure 2: Training progress and loss values across epochs during the fine-tuning of the CLIP model

Observation

The output shows the loss values for each epoch during the fine-tuning of the CLIP model. The loss decreased from 2.4056 in Epoch 1 to 2.0479 in Epoch 3, indicating improved model performance over the training process. The fine-tuned model was successfully saved.

Step 3 - Similarity Metrics :

Compute Image Embeddings for Retrieval

- Image embeddings were computed for the entire dataset using the CLIP model. The dataset was loaded in batches, and for each batch, the `pixel values` were passed through the model to extract the image features.
- The extracted features were normalized, and embeddings for the entire dataset were gathered in two lists: one for the embeddings and one for the indices.
- The embeddings were then concatenated to form a complete set, which was later used for image retrieval.

Encode Query and Compute Similarity

- The query was encoded using the CLIP model to generate its text embeddings. The processor was used to prepare the text input, and the embeddings were normalized to ensure consistency.
- The similarity between the text query and the image embeddings was then computed using different methods such as cosine, dot product, or Euclidean distance. The similarity score was used to rank the relevance of images to the text query.

Retrieve Image Function

- The retrieve_images function was used to retrieve images based on a text query. First, the query was encoded using the CLIP model, and similarity scores between the query and image embeddings were computed using a specified similarity method (e.g., cosine).
- The top k most similar images were selected, and their corresponding captions and scores were stored. The images were then displayed with their captions and similarity scores.
- The results were also visualized in a bar plot, displaying the similarity scores for the top k images.

Step 4 -Evaluation Metrics :

Ranking-based Evaluation Metrics

- The text embeddings and image embeddings were used to compute the similarity matrix using different methods like cosine, dot product, and Euclidean distance. The similarity_matrix function calculated similarity scores between text and image embeddings.
- The evaluate_ranking function was used to compute various ranking metrics, including Hit@K, MRR (Mean Reciprocal Rank), Mean Rank, Median Rank, and NDCG@K. These metrics were essential to evaluate the performance of the model in retrieving relevant images based on text queries.

Compare Metrics Across Val/Test for Similarity Methods

- Different similarity methods were evaluated across both the validation and test datasets. For each method, the evaluation metrics were calculated using the evaluate_ranking function.

Final Ranking Metrics:

	cosine		dot		euclidean	
	Validation	Test	Validation	Test	Validation	Test
Hit@1	0.320	0.344	0.320	0.344	0.320	0.344
Hit@5	0.600	0.672	0.600	0.672	0.600	0.672
Hit@10	0.704	0.792	0.704	0.792	0.704	0.792
MRR	0.454	0.479	0.454	0.479	0.454	0.479
MeanRank	9.248	9.464	9.248	9.464	9.248	9.464
MedianRank	4.000	3.000	4.000	3.000	4.000	3.000
NDCG@1	0.320	0.344	0.320	0.344	0.320	0.344
NDCG@5	0.467	0.507	0.467	0.507	0.467	0.507
NDCG@10	0.502	0.547	0.502	0.547	0.502	0.547

Figure 3: Figure of the Ranking Metrics Table.

Observation from the Ranking Metrics Table

- **Cosine** and **dot** similarity methods showed nearly identical performance across all metrics, with **Euclidean** being slightly lower in performance.
- **Hit@1** and **MRR** were consistently higher for **cosine** and **dot**, while **Euclidean** had marginally lower values.
- Overall, the ranking quality was similar for all methods, with **Euclidean** performing a bit worse in higher ranks.

Plotting Evaluation Metrics Comparison

- The overall observation from the Ranking Metrics Table was plotted as bar charts. The **validation** and **test** results were compared side by side for each method, using different **hatch patterns** to distinguish between the two datasets.

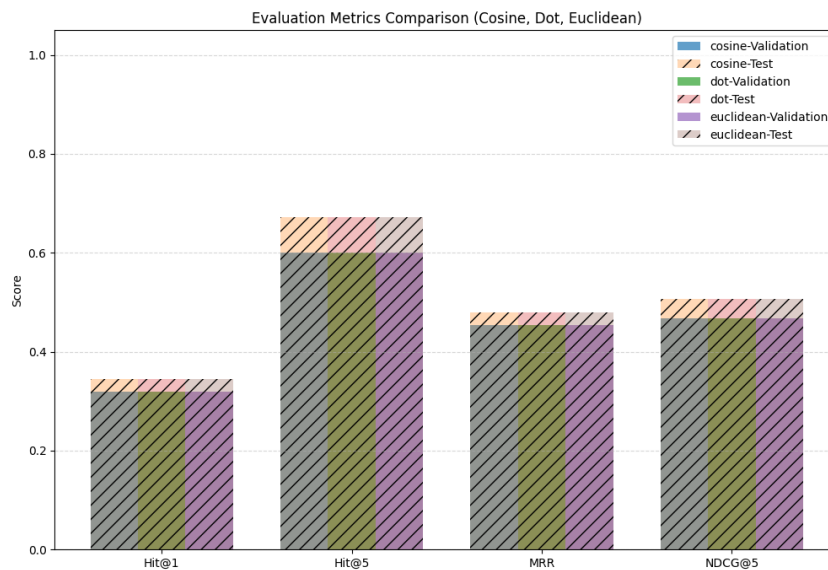


Figure 4: Evaluation Metrics Comparison Plot.

This bar chart compares the evaluation metrics (Hit@1, Hit@5, MRR, NDCG@5) across three similarity methods (Cosine, Dot, Euclidean) for both the Validation and Test datasets. Cosine and Dot methods show similar performance across metrics, while Euclidean performs slightly lower, especially for Hit@1 and MRR.

Step 5 -Image Retrieval System: Final Output and Analysis

Query-based Image Retrieval

- Finally, the `retrieve_images` function was used to retrieve images based on a **text query**. The query was first used to retrieve the top 5 images from the **validation set** and then from the **test set**. The **cosine similarity** method was applied to find the most relevant images for the given query.
- The results showed the top-ranked images along with their captions and similarity scores.

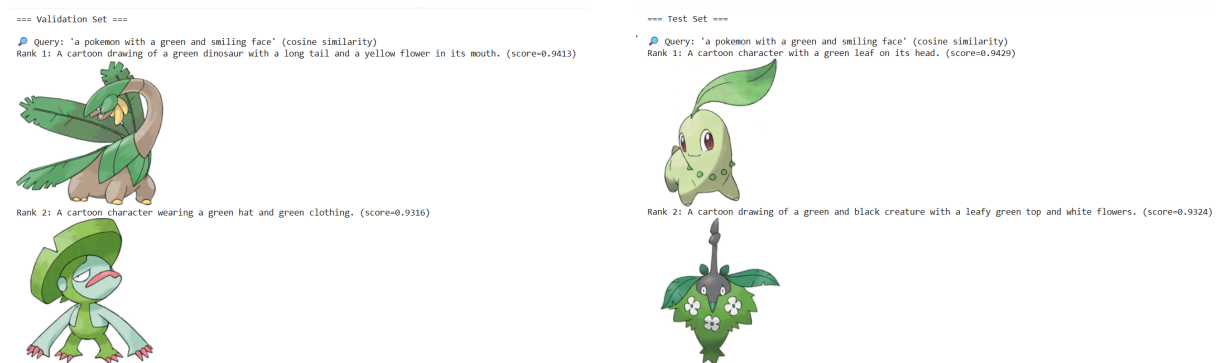


Figure 5: Top 2 Results from the Validation Set and Test Set.

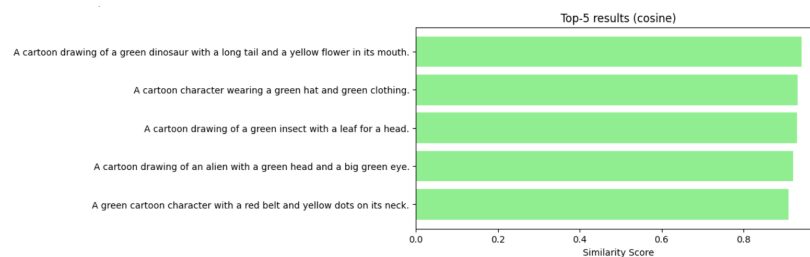


Figure 6: Top 5 results with similarity score from the Validation Sets

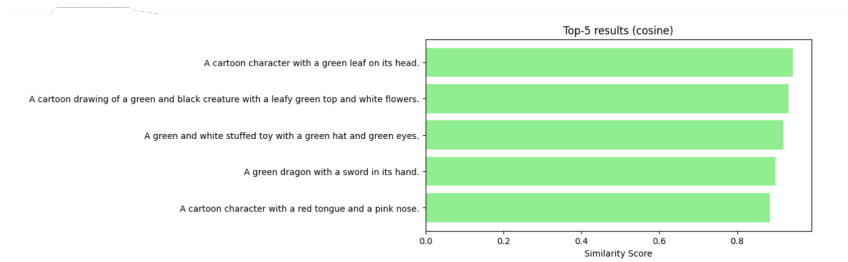


Figure 7: Top 5 results with similarity score from the Test Sets

The bar charts show the top 5 results from both the **Validation** and **Test Sets** for the query "a pokemon with a green and smiling face," using **cosine similarity**. The scores, ranging from 0 to 1, indicate how closely the images match the query, with higher scores representing better matches.

Performance Analysis :

Challenge :

Possible Improvements/Future work/Future Upgrades May Include :

Conslusion :

- The **Text-Guided Image Retrieval System** has been successfully implemented and tested.
- Upon entering a query, the system retrieves the top 5 images from the dataset, ranked by **cosine similarity**.
- The images, along with their similarity scores, are displayed alongside a bar chart that shows their relevance to the query.
- The high similarity scores for the top-ranked images confirm the system's effectiveness in retrieving the most relevant images.
- The **Validation** and **Test** sets both show consistent and reliable results, validating the system's robustness.
- Overall, the system meets its functional requirements and demonstrates strong performance in matching images to textual queries, making it a successful implementation.

Reference:

1. [Kaggle Dataset - pokemon-llava-images-and-text-descriptions](#)